

10 - OO Design

Nel design della programmazione OO adottiamo una **prospettiva software**, dove gli elementi di un modello corrispondono agli elementi di un sistema software orientato agli oggetti ed è **indipendente dai vari linguaggi di programmazione**.

Per descrivere le relazioni tra oggetti e classi presenti in un sistema costruito con metodologia OO si usa il **diagramma delle classi con prospettiva software**.

Classe

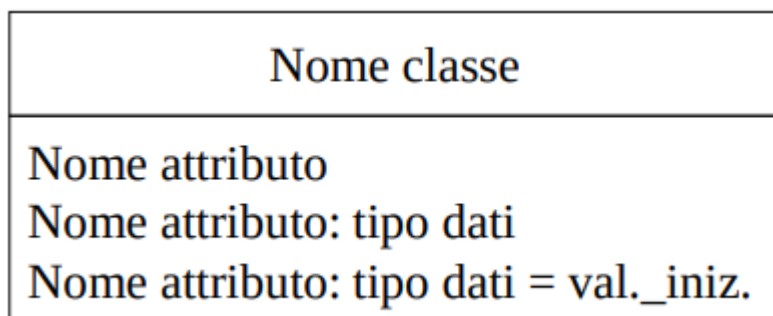
```
public class Dialler{
}
```

Si rappresenta tramite il seguente disegno madre



Attributi

Gli attributi generalmente sono **privati** e rappresentano le **proprietà strutturali** degli oggetti di una classe.



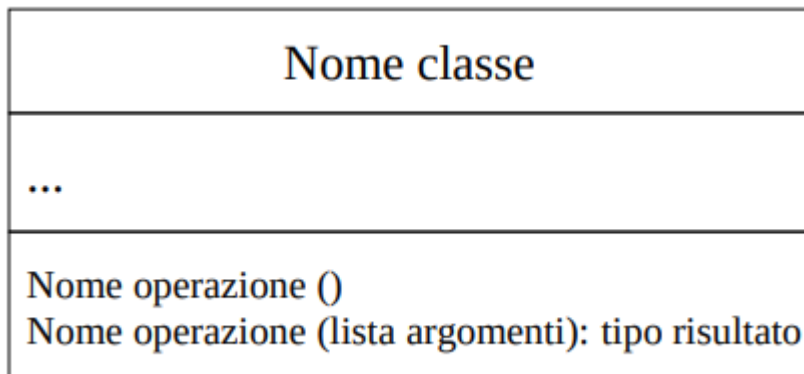
In un linguaggio OO, gli attributi corrispondono al nome di una **variabile di istanza** della classe.

```
public class Dialler{
    private Vector digits;
    int nDigits;
    public void digit(int n);
}
```

```
protected boolean recordDigit(int n);
}
```

Operazioni

Sono mostrate in UML nel segmento inferiore e disegnate nel seguente modo:



Rappresentano le **azioni** che si possono eseguire sugli oggetti di una classe. A differenza degli UML precedenti, nel diagramma delle classi è utile e doveroso mostrare i nomi delle operazioni, dato che sono i lavori principali di una classe.

Nei linguaggi OO queste azioni corrispondono ai **metodi** della classe.

La visibilità delle operazioni è per default **pubblica**, se non lo sono conviene specificarlo con – nell'UML o possono addirittura essere **protette**, dimostrabili tramite #, ma è un caso abbastanza particolare da gestire.

Nell'UML generalmente non si mostrano le **operazioni basilari**, come il `GetAtt` o il `SetAtt`, o anche il **costruttore stesso**, dato che è facilmente intuibile la loro presenza nel codice. Rendendo così vantaggiosa la lettura dell'UML e avendo uno schema più compatto.

Responsabilità di una classe



Classe	Responsabilità
Sale	• Sa calcolare il totale della vendita • Conosce le righe di vendita della vendita
SalesLineItem	• Sa calcolare il totale parziale della riga di vendita • Conosce il prodotto della riga di vendita
ProductDescription	• Conosce il prezzo del prodotto

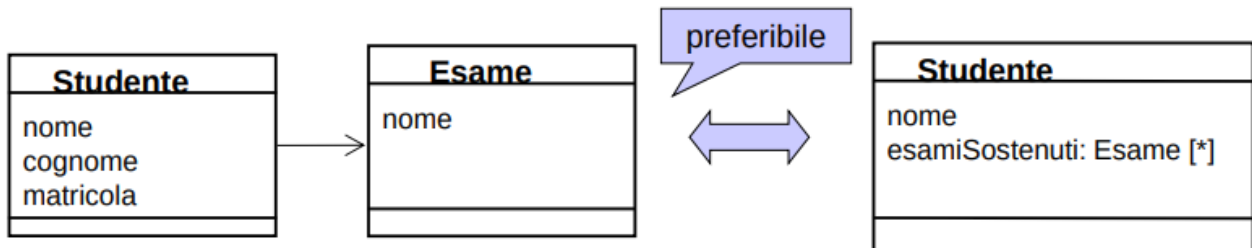
La responsabilità della classe serve per poter capire, tramite brevi commenti, cosa andranno a fare le funzioni che si eseguono all'interno della classe.

Associazioni

Le associazioni tra più classi vengono rappresentate nel diagramma delle classi tramite un collegamento tra un attributo di una classe e lo stesso attributo di appartenenza in un'altra classe.

Le associazioni hanno un verso di navigazione. Il verso della freccia ci fa capire quale classe conosce il tipo presente nell'altra classe.

L'associazione è una rappresentazione alternativa della **proprietà strutturale**, il tipo base presente in una classe viene quindi cambiato in un'altra classe con le sue informazioni.

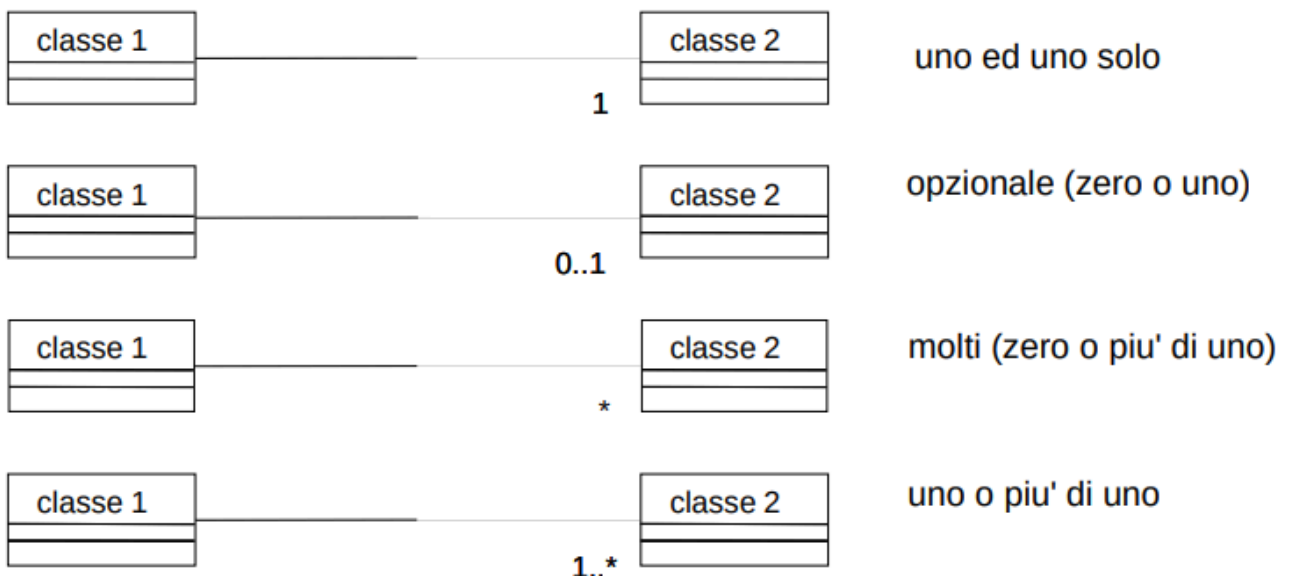


Molteplicità

⚠ Nota

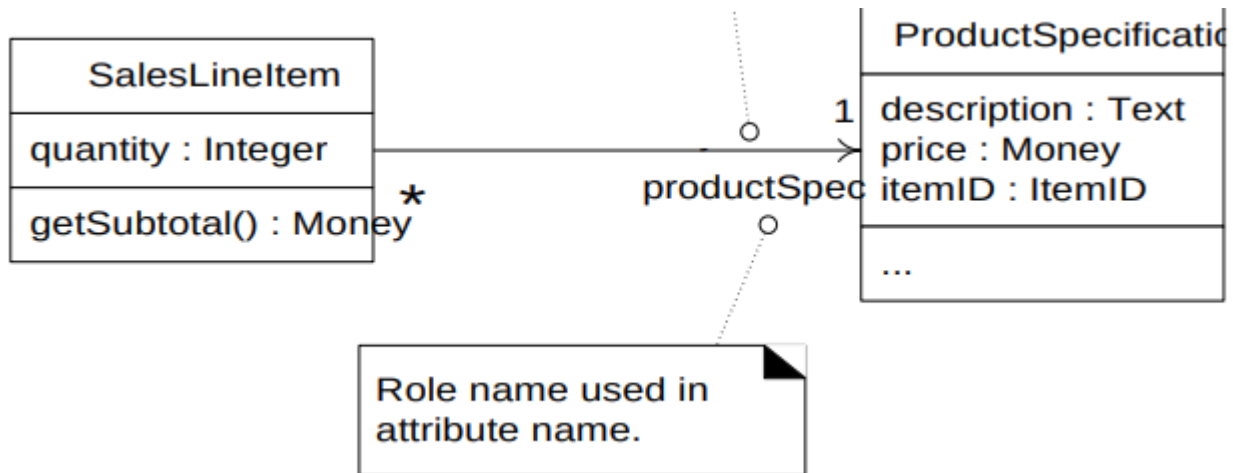
Alcuni professori preferiscono la notazione con le frecce nelle associazioni di molteplicità e non i vari pallini

La molteplicità specifica quante istanze di una classe possono essere associate con una singola istanza di un'altra classe. Il simbolo di molteplicità adiacente alla classe 2 indica quante istanze della classe 2 possono avere legami con una singola istanza della classe 1.



Ruoli

Non è possibile definire dell'etichette alle associazioni ma si possono definire dei **ruoli**. Il ruolo indica in modo esplicito il nome della proprietà di una classe rappresentata come associazione.



Dove `productSpec` è il seguente codice:

```

public class SalesLineItem
{
    ...

    private int quantity;

    private ProductSpecification productSpec;
}
  
```

Dipendenza

La dipendenza in UML delle classi viene rappresentata tramite **freccia tratteggiata** per distinguerla dall'associazione.

Si ha una dipendenza tra classi quando una classe **è a conoscenza di un'altra classe**, dove il cambiamento della classe *A* può modificare la classe *B*, che dipende da essa (questo principio vale anche per le associazioni e le generalizzazioni).

Come nelle associazioni, l'oggetto fornitore deve essere visibile all'oggetto cliente

- Associazione



- Dipendenza



```

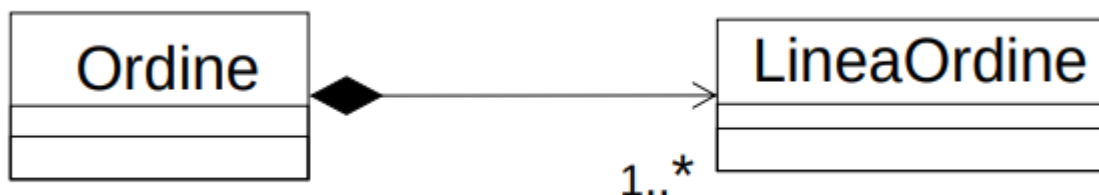
public class A {
    private B itsB;
    public void f() {
        itsB.f();
    }
}
  
```

Aggregazione e composizione

L'aggregazione non ha un significato distinto da una generica associazione e quindi non c'è bisogno di usarla.

A livello teorico però, la composizione è un'associazione in cui:

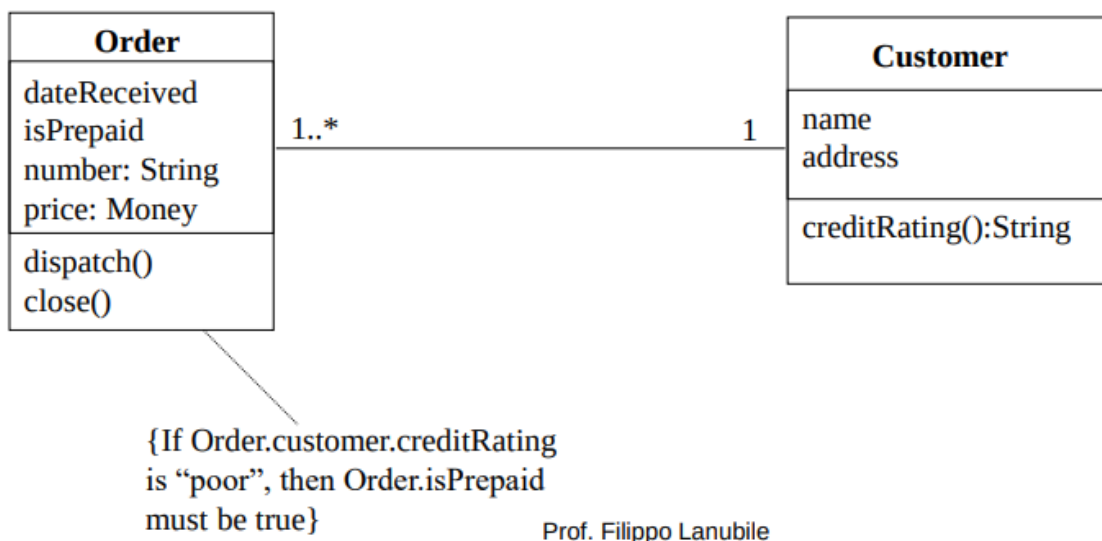
- L'oggetto contenitore è l'unico responsabile della creazione degli oggetti contenuti
- La cancellazione dell'oggetto contenitore ha come effetto la cancellazione degli oggetti contenuti
- La copia dell'oggetto contenitore ha come effetto la copia degli oggetti contenuti



Vincoli

Rappresentano la possibilità di inserire qualche dettaglio agli elementi di un diagramma delle classi.

Si possono esprimere in modo informale tra **parentesi graffe** e **Object Constraint Language (OCL)** è un linguaggio formale per specificare i vincoli.



Generalizzazione

Molto più importante dei concetti precedenti è proprio la rappresentazione grafica delle **generalizzazioni**.

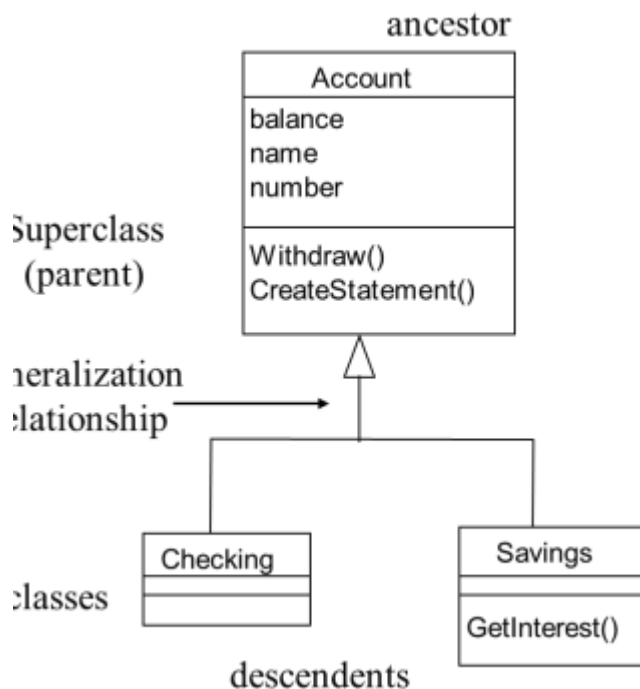
Questo concetto ci permette di individuare la **condivisione delle caratteristiche delle classi**, ove le caratteristiche sono operazioni o attributi.

Gli oggetti delle **sottoclassi** ereditano le stesse caratteristiche delle **superclasse** (classe generale).

Una sottoclasse può definire delle **estensioni**, ovvero caratteristiche aggiuntive e possono eseguire un **overriding**, ossia una modifica dell'implementazioni delle operazioni ereditata.

✎ Uso comune dell'overriding

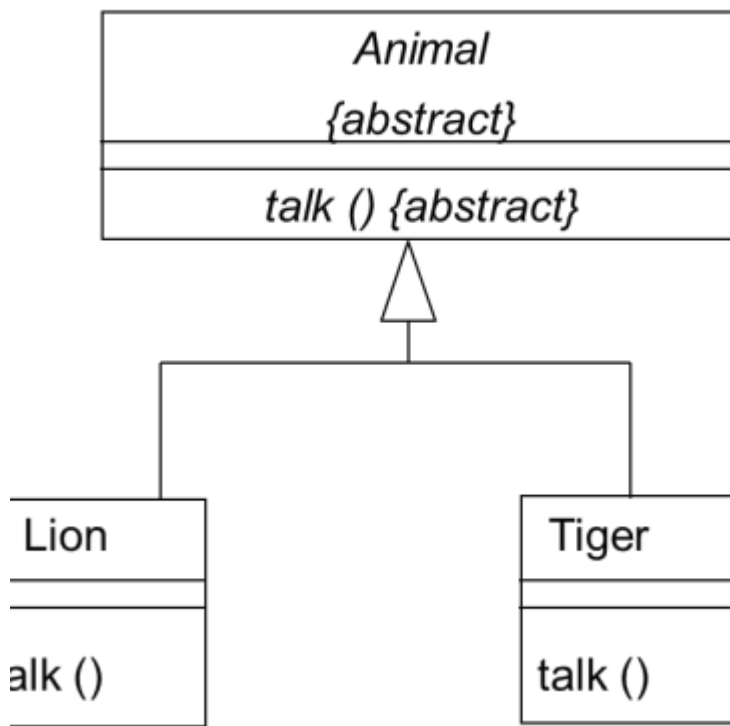
Molto spesso l'overriding anche se permesso non è la soluzione migliore e ben vista per risolvere i problemi di modifica delle operazioni per le classi figlie



Classi astratte e concrete

Una classe astratta è una classe che non può essere direttamente istanziata e contiene delle operazioni **astratte**, prive di implementazione che vengono implementate dalle classi figlie in base all'uso che ne devono fare.

Una sottoclasse di una classe astratta viene definita **classe concreta** ed è l'unico metodo in cui si possono implementare le operazioni della classe astratta e istanziarne gli oggetti.



Le classi astratte hanno portato all'uso delle **interface**.

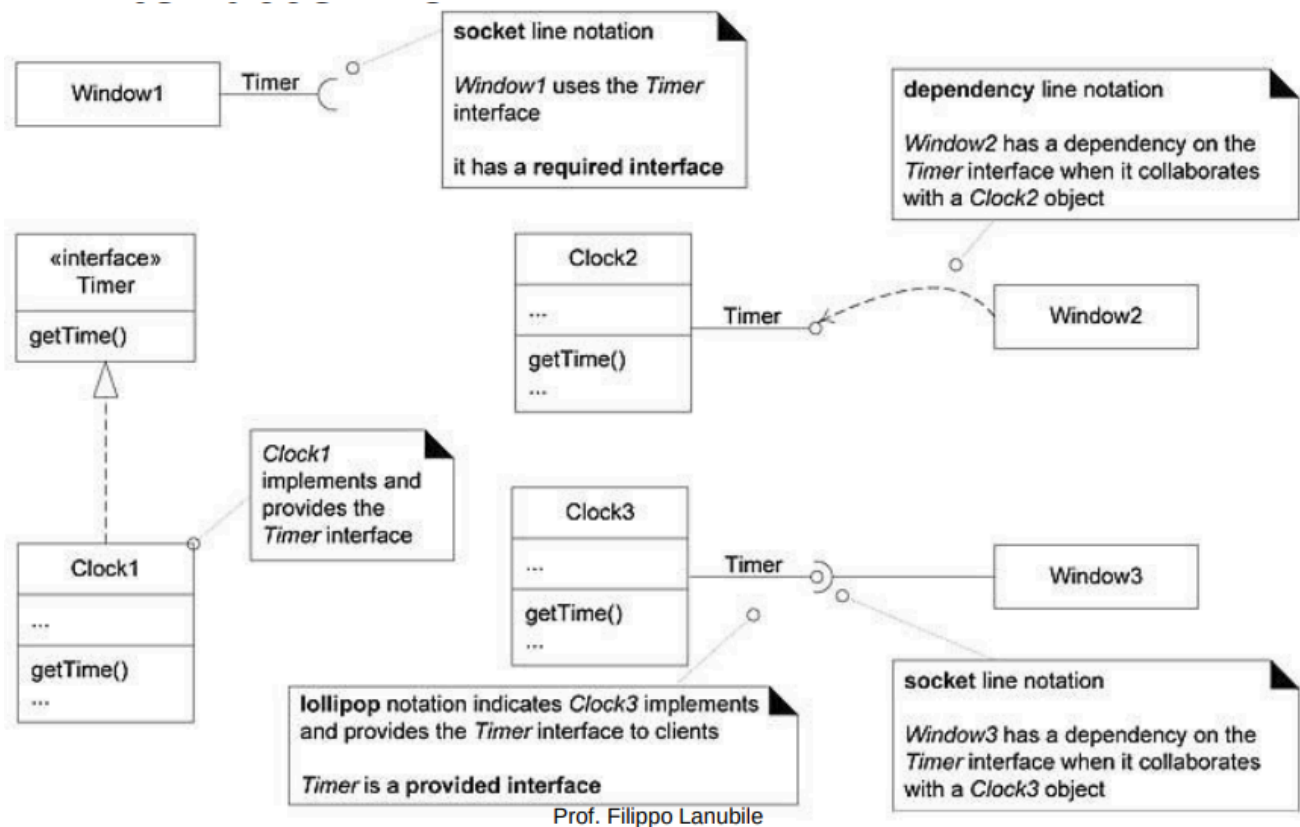
Interface

Un'interfaccia è un costrutto puro basato sulla classe astratta in grado di estendere una o più interfacce.

Una classe **realizza** un'interfaccia se è sostituibile a essa, poiché ne implementa una o più operazioni astratte.

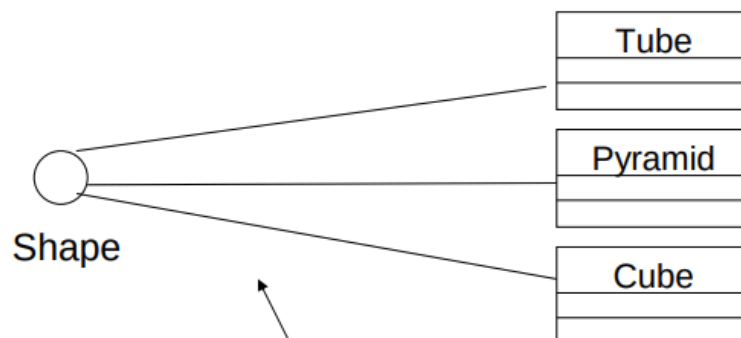
Una classe **richiede** una sua interfaccia se necessita delle sue operazioni per funzionare, si crea una dipendenza da essa e una gerarchia d'ordine.

In UML esistono diversi modi per rappresentare un Interface e chi la richiede:



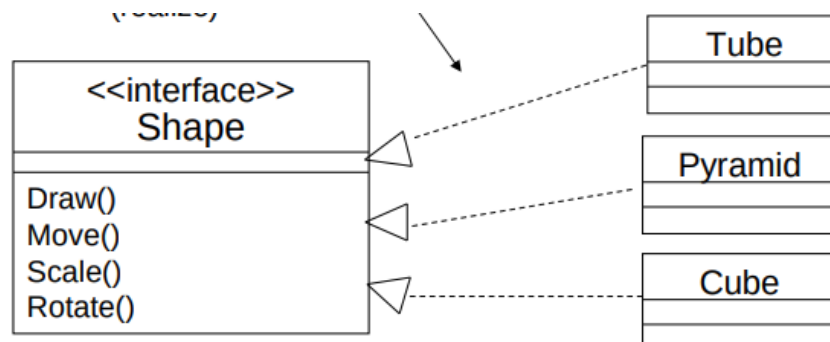
In passato si rappresentava in maniera più semplice tramite la **rappresentazione lollipop**:

Rappresentazione
iconica
("lollipop")



o tramite quella **conica**:

Rappresentazione
canonica
(Class/Stereotype)



Prof. Filippo Lanubile

Nessuno dei due metodi però permette di identificare chi ha richiesto l'interfaccia.

Costruzione di diagrammi delle classi con prospettiva software funzionanti

Queste sono le regole generali da seguire per avere un ottimo diagramma delle classi funzionante e dimostrabile:

- **Non cercare di modellare tutto il sistema con un singolo diagramma**
- Ogni diagramma deve avere un suo criterio:
 - Diagramma che mostra le classi che collaborano nella realizzazione di una user story
 - Diagramma dei package
 - Diagramma che mostra la gerarchia di classi
- Non è importante distinguere correttamente tra associazioni e dipendenza ma è **fondamentale** indicare **precisamente la direzione dei riferimenti ad altre classi**. Poiché crea confusione e dà il senso di non conoscere quale classe richiede informazioni da un'altra classe.
- Principio di **parsimonia**, non mostrare tutto ciò che è presente nel codice ma solamente quelle significative per il diagramma in questione.

Modellare il comportamento

Per modellare il comportamento di un progetto software si fa ricorso alla **documentazione** del comportamento di un gruppo di oggetti che realizzano un singolo scenario di utilizzo del sistema. Tale comportamento è visualizzabile tramite il **diagramma di sequenza**.

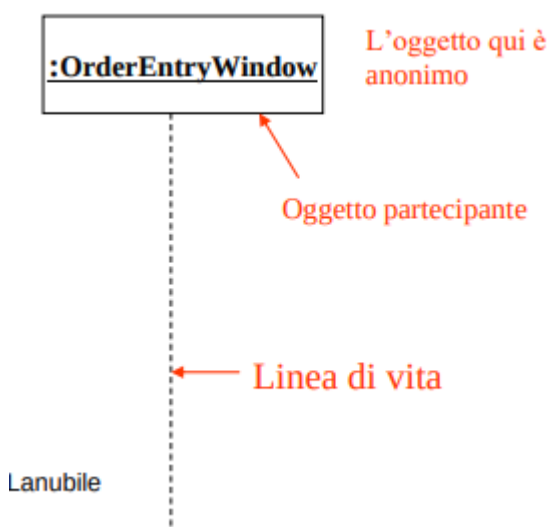
Diagramma di sequenza

Il diagramma mostra l'interazione tra oggetti con enfasi sulla sequenza dei messaggi. Nel diagramma di sequenza si hanno vari elementi, primi tra tutti sono i **partecipanti**.

Personaggi

I **partecipanti** del diagramma, i quali sono **oggetti** rappresentati come un rettangolo in cima a una linea tratteggiata.

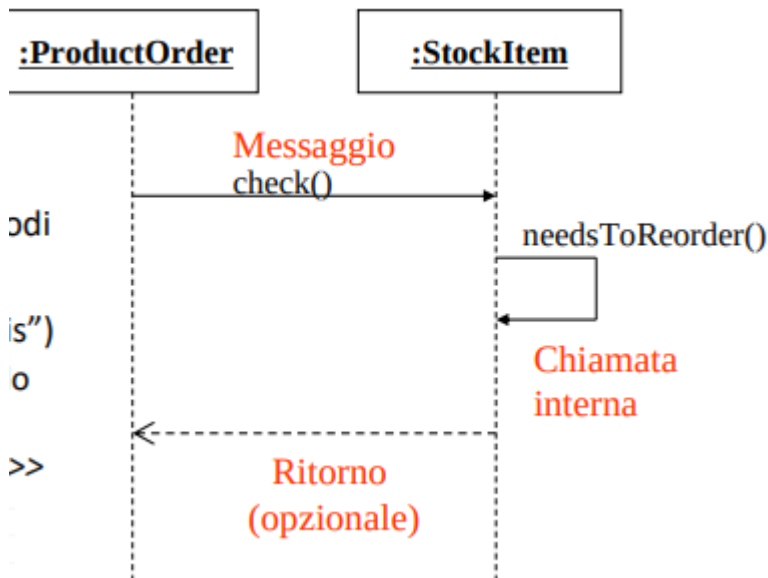
La linea tratteggiata rappresenta **lo scorrere del tempo** detta **Linee di vita**.



Messaggi

Nel diagramma di sequenza esistono anche i **messaggi**, ovvero la comunicazione tra oggetti realizzata tramite la chiamata dei metodi.

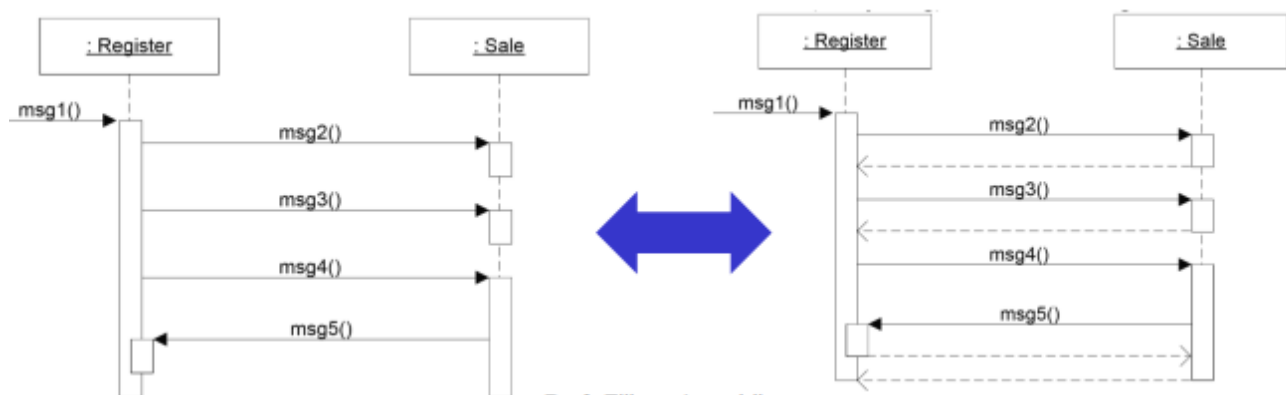
I messaggi possono essere anche **speciali**, come il **create**, il **return** o la **destroy** di un oggetto.



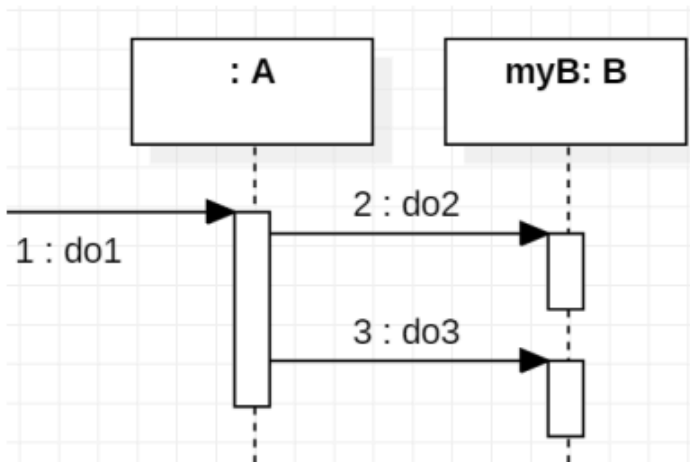
Barre di attivazione

Le barre di attivazione sono sottili **rettangoli** posti sulla linea di vita di un oggetto e servono ad indicare quando il partecipante è attivo nell'interazione.

La presenza di questo elemento nel diagramma rende ridondante l'uso delle linee di **restituzione del controllo**.



Esempio di diagramma di sequenza e codice Java corrispondente

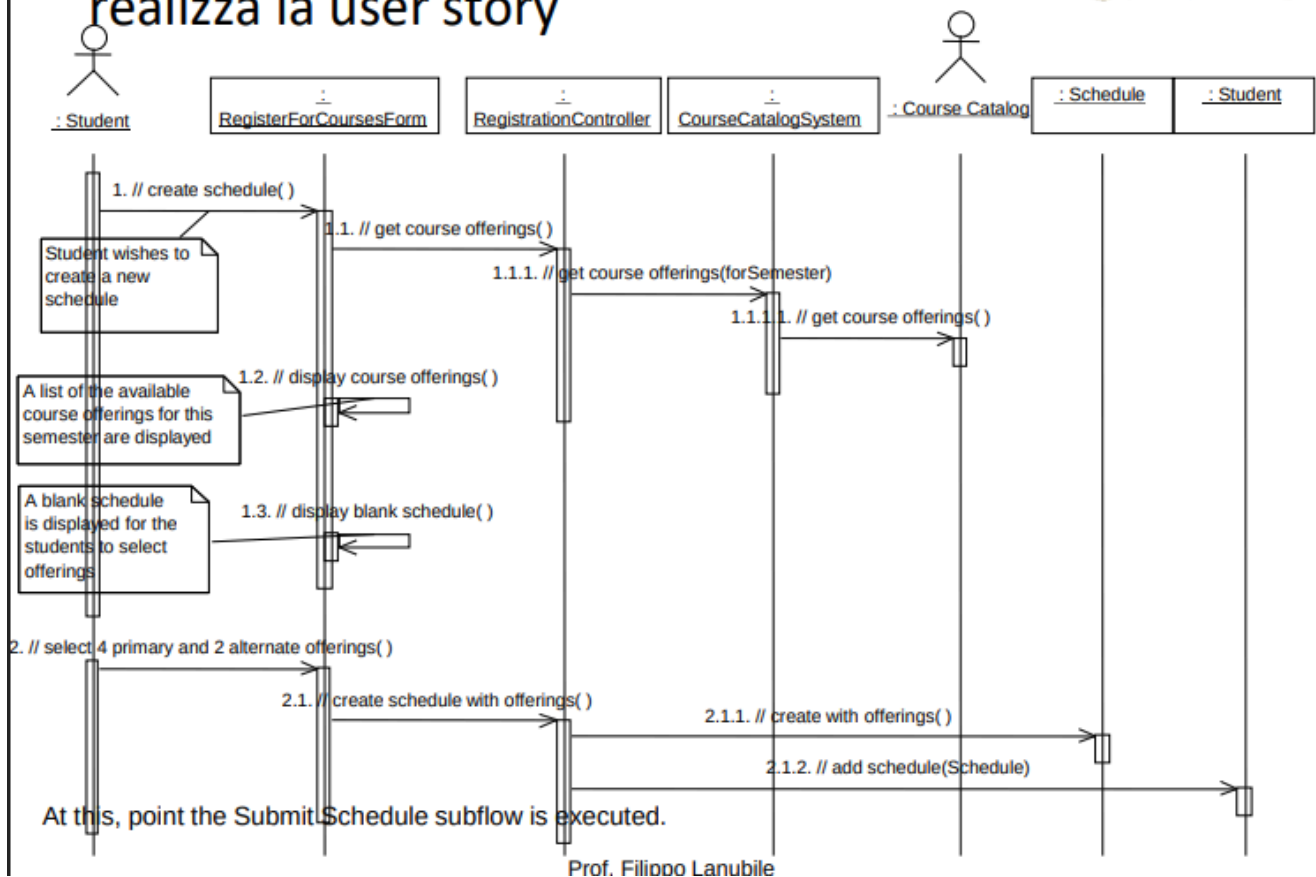


```

public class A {
    private B myB = ...;

    public void do1() {
        myB.do2();
        myB.do3();
    }
}
  
```

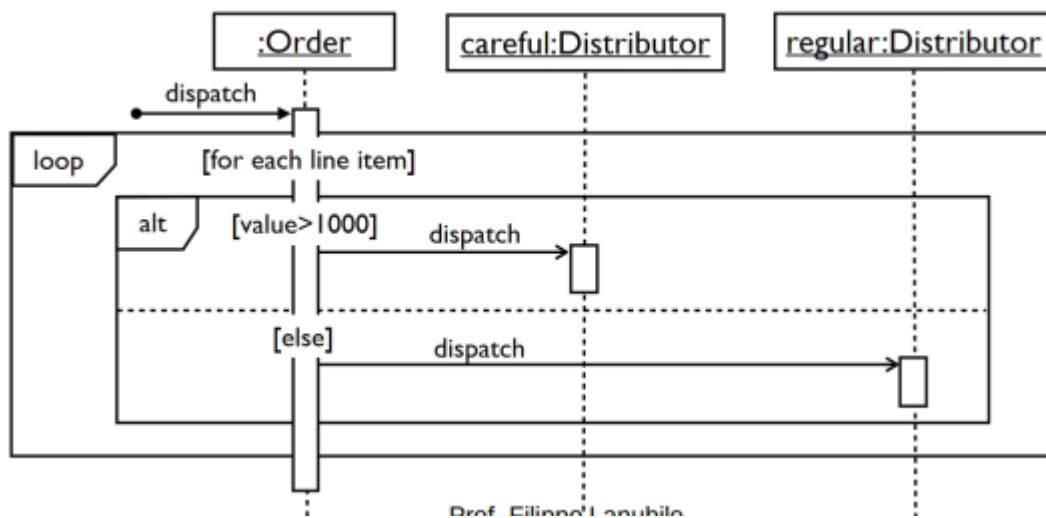
Esempio di diagramma di sequenza che realizza la user story



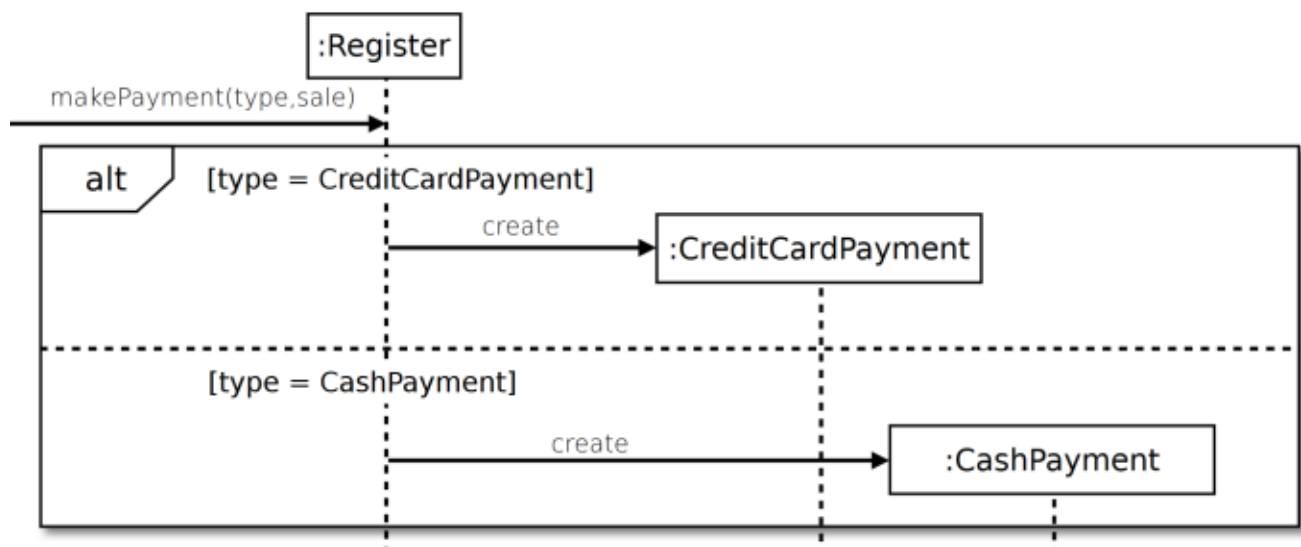
UML 2

In **UML 2** esistono dei simboli che non era possibile utilizzare prima, detti **frame di interazione**. Il frame di interazione è una cornice intorno ad una parte del diagramma di sequenza, tra gli elementi principali vi si ha:

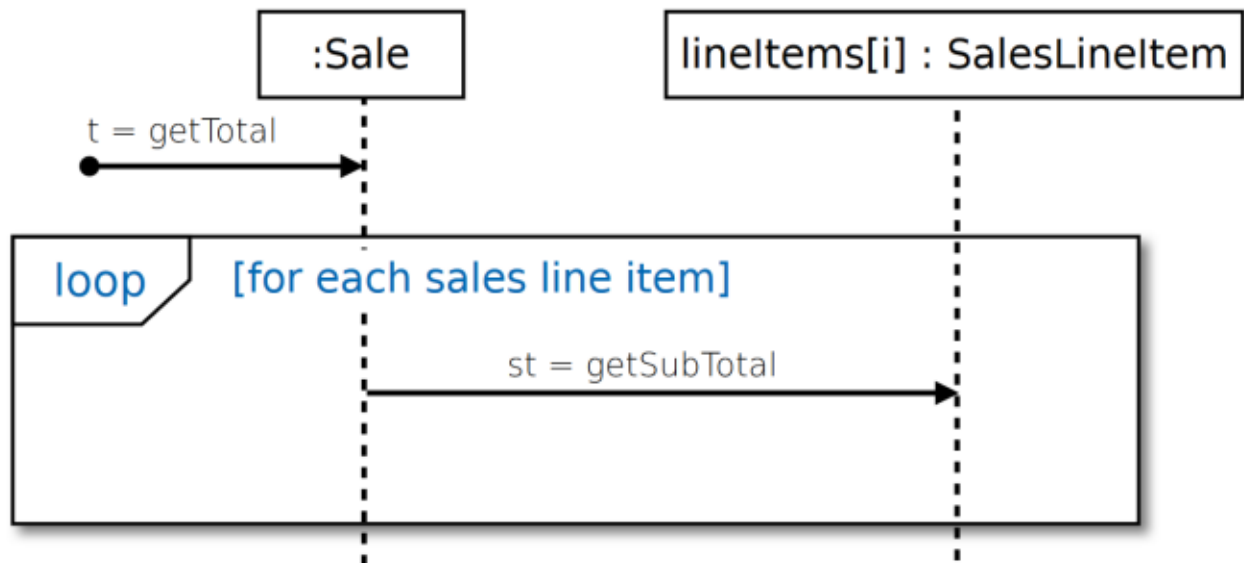
- **opt** che sostituisce gli **if**
- **alt** che serve per **l'else**
- **loop** che serve per i cicli iterativi



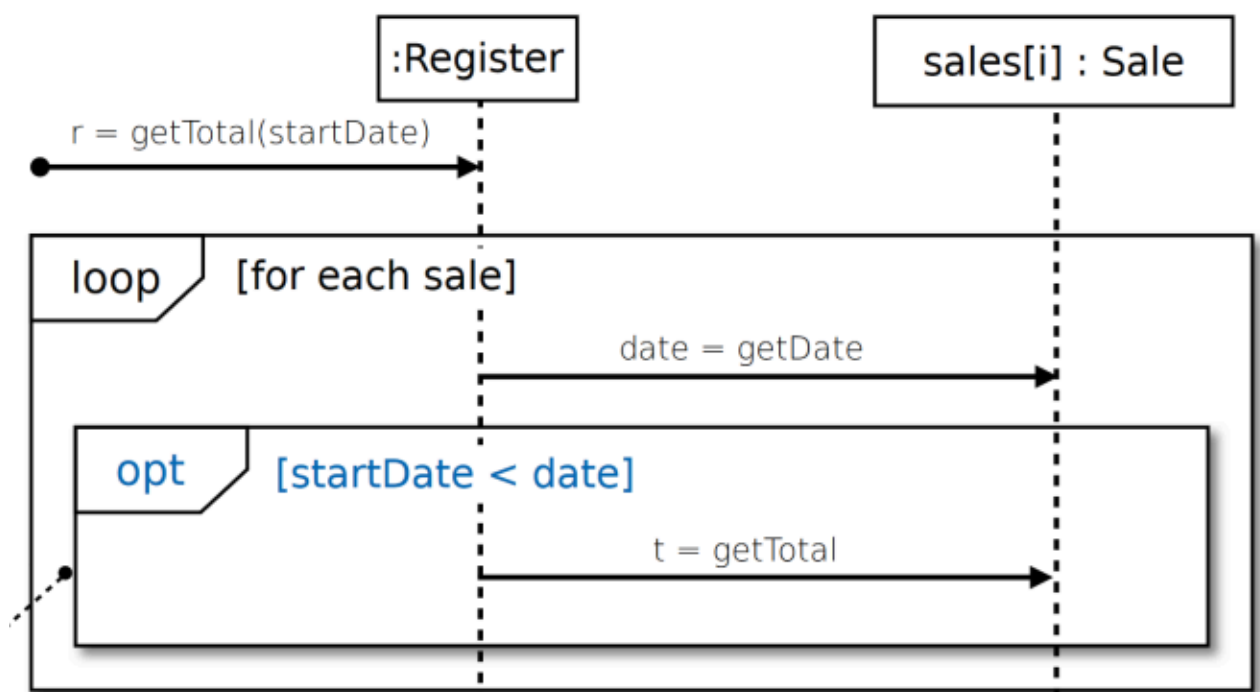
Esempio Alt:



Esempio loop:



Esempio opt:



Attività dell'OO design

L'attività di OO design serve per valutare altre vie percorribili nella realizzazione del sistema, analizzando i requisiti, i moduli, ecc...

Serve specialmente per tenere traccia di cosa si è fatto e rispondere alla domanda: "cosa si è fatto qui?", infatti la **documentazione** è fondamentale in questa fase per portare traccia degli avanzamenti e le modifiche.

Avviene il riuso del **design pattern**, delle librerie e del **framework OO** e su quest'ultimi si applica il principio di OO design per evitare il **debito tecnico**.

Principi di OO desing

I principi dell'OO desing sono:

- Information hiding
- Alta coesione
- Basso accoppiamento
- Presentazione separata
- Do Not Repeat Yourself (**DRY**)
- **SOLID method**

Ogni elemento di conoscenza deve avere una sola, non ambigua, autorevole rappresentazione all'interno di un sistema

Qualsiasi parte significativa di un metodo dovrebbe essere implementato in unica zona apposita del codice sorgente, evitando **cloni** dati dal **copy-paste** e creando astrazioni in caso ci dovessimo imbattere in ripetizioni del concetto.

SOLID

Single Responsibility	Non deve esistere più di una ragione per cui una classe debba essere modificata 1 classe → 1 responsabilità
Open/Closed	Le entità della programmazione devono essere aperte alle estensioni e chiuse alla modifica Classi/Moduli/Funzioni estendibili
Liskov Substitution	Le sottoclassi devono poter essere utilizzate al posto della loro classe padre Uso corretto della generalizzazione
Interface Segregation	I Client non devono dipendere da interfacce che non usano Interfacce specializzate per ogni client sono migliori di una singola grossa interfaccia
Dependency Inversion	Moduli di alto livello non devono dipendere da moduli di basso livello, entrambi devono dipendere da astrazioni Le astrazioni non devono dipendere dai dettagli, ma viceversa